ANALYSIS

Feature Gap Analysis

Requested Features vs. Current State

PREPARED FOR	PREPARED BY	DATE	DOCUMENT
Precision Structures Inc.	EBYG Automation — SSC Digital Services	March 19, 2026	02 of 17

Prepared: March 18, 2026 **Auditor:** SSC Digital Services **Subject:** Requested features mapped to implementation details **Companion document:** Precision Site Audit

Overview

This document maps each feature discussed for the Precision Structures website to a concrete implementation recommendation. Each feature is assessed for business outcome, effort, risk, dependencies, and whether it is a base requirement (needed for a credible launch) or an add-on (enhances the site after launch).

Effort scale:

- **S (Small):** Less than 1 day of focused work
- **M (Medium):** 1-5 days of focused work
- **L (Large):** 1-3 weeks of focused work

Risk levels:

- **Low:** Straightforward implementation, minimal chance of failure or rework
- **Medium:** Some unknowns or dependencies that could cause delays
- **High:** Significant complexity, external dependencies, or potential for scope creep

Feature Map

#	FEATURE	BUSINESS OUTCOME	RECOMMENDED SOLUTION	EFFORT	RISK	DEPENDENCIES	BASE OR ADD-ON
1	Page Review PDFs	Client can review site pages offline; creates an approval paper trail	Playwright script that navigates each page, captures full-page screenshots or PDF renders, outputs to a `/reviews` folder	**S**	Low	Node.js installed locally; Playwright npm package	Add-On
2	Final Site Changes + Launch	Site is polished, accurate, and ready for public promotion	Direct HTML/CSS edits on `develop`, review on staging, merge to `master` for production deploy	**S**	Low	Completed content review; real gallery photos; contact form fix	**Base**
3	Client Update Portal	Non-technical staff can update content (contact info, service descriptions, gallery) without developer involvement	Lightweight admin: either Decap CMS (Git-based, no database) or a custom minimal UI that edits a JSON content file and triggers a rebuild	**L**	Medium	Templating layer to separate content from markup; Git-based workflow or API; user authentication	Add-On
4	AI Chatbot (partially built)	Visitors get immediate answers to common questions; differentiates the brand; captures lead intent	Harden existing `chatbot.js` implementation: (1) ensure backend proxy is stable, (2) write a Precision-specific system prompt with company persona, (3) add conversation logging for analytics, (4) add fallback for API downtime	**M**	Medium	Backend proxy endpoint (currently exists); Claude API access; defined company persona document	Add-On
5	Company Voice / Persona	Consistent brand tone across website copy, chatbot responses, social media, and client communications	Create a brand voice document defining tone (professional, straightforward, confident), vocabulary preferences, audience assumptions, and example phrasings. Encode as a chatbot system prompt.	**S**	Low	Stakeholder input on brand identity; review of existing marketing materials	**Base**
6	Social Media Support	Consistent online presence; local brand awareness; referral traffic to website	Content calendar (monthly), post templates for project completions / industry tips / team highlights, workflow for creating and scheduling posts using existing Facebook and Instagram accounts	**M**	Low	Real project photos; brand voice document; access to social media accounts; scheduling tool (e.g., Buffer, Later, or manual posting)	Add-On
7	SEO Technical	Higher visibility in local search results,	(1) Add JSON-LD structured data —	**M**	Low	Google Search Console	**B**

#	FEATURE	BUSINESS OUTCOME	RECOMMENDED SOLUTION	EFFORT	RISK	DEPENDENCIES	BASE OR ADD-ON
	Setup	more organic traffic; better Google Business Profile integration	LocalBusiness, Service, and FAQPage schemas. (2) Add canonical tags to all 6 pages. (3) Update sitemap.xml with current dates. (4) Register Google Search Console. (5) Optimize meta descriptions for target keywords.			access; Google Business Profile ownership; target keyword research	
8	Email-to-Staging Update Workflow	Client sends an email describing a desired change; the change is automatically drafted on staging for review	Email intake (dedicated mailbox or forwarding rule) → Azure Function parses email content with AI → generates proposed HTML diff → commits to `develop` → notifies developer for review	**L**	High	Azure Functions; email service integration (SendGrid inbound parse or Microsoft Graph API); AI parsing logic; review/approval mechanism; error handling for ambiguous requests	Add-On
9	Security + Testing	Reduced vulnerability surface; consistent quality; accessibility compliance	(1) Add Content Security Policy header. (2) Implement form backend with CAPTCHA. (3) Run WAVE or axe accessibility audit and fix issues. (4) Add automated link checking to CI/CD. (5) Add basic visual regression testing.	**M**	Low	Azure SWA config access; form backend implementation; CI/CD pipeline modification	**Base**
10	Analytics / Reporting	Data-driven understanding of visitor behavior, traffic sources, and conversion rates	Install Google Analytics 4 (GA4): add tracking snippet to all pages, configure conversion events (form submission, chatbot interaction, phone click), set up a basic dashboard	**S**	Low	Google Analytics account; access to add script tags to all pages	**Base**
11	Contact Form Backend	Form submissions actually reach the business; leads are captured; visitors receive confirmation	Azure Function HTTP trigger: receives form POST, validates input, sends notification email via SendGrid, stores submission record, returns success/failure. Add CAPTCHA (hCaptcha or reCAPTCHA) to prevent spam.	**M**	Low	Azure Functions plan (can use free tier); SendGrid account (free tier: 100 emails/day); DNS records for email sender verification	**CRITICAL — Base**

Detailed Feature Assessments

1. Page Review PDFs

Current state: No mechanism exists to generate page review artifacts.

Recommended approach:

- Install Playwright as a dev dependency
- Write a Node.js script that visits each of the 6 pages on the staging URL
- Capture full-page PDF or PNG for each page at desktop and mobile widths
- Output files with timestamps to a local `/reviews` directory
- Optionally commit to a `reviews` branch or upload to shared storage

Deliverable: A single script file (`scripts/generate-reviews.js`) and a package.json with the Playwright dependency.

Why this approach: Playwright is headless, fast, and produces pixel-accurate renders. No paid service needed. The script can be rerun anytime a review cycle is needed.

---

2. Final Site Changes + Launch

Current state: Site is functional on staging but has known issues (contact form, copyright, gallery placeholders, stale sitemap).

Pre-launch checklist:

TASK	STATUS
Contact form connected to backend	Blocked (needs Feature #11)
Copyright updated to 2026	Not started
Gallery placeholders replaced with real photos	Blocked (needs photos from client)
Sitemap dates updated	Not started
Analytics installed	Not started (Feature #10)
JSON-LD structured data added	Not started (Feature #7)
Canonical tags added	Not started (Feature #7)
Final content review and copy edits	Not started
Cross-browser testing (Chrome, Firefox, Safari, Edge)	Not started
Mobile testing on real devices	Not started
Merge `develop` to `master` for production deploy	Blocked (all above)

This feature is a coordination milestone, not a build task. It depends on several other features being completed first.

3. Client Update Portal

Current state: All content is hardcoded in HTML. No admin interface exists.

Option A — Decap CMS (recommended for this project):

- Git-based CMS that runs in the browser
- No database, no server — edits commit directly to the GitHub repo
- Supports content fields, image uploads, and preview
- Free and open source
- Requires a templating layer (Eleventy or similar) to separate content from markup

Option B — Custom JSON + Admin UI:

- Store site content in a `content.json` file
- Build a simple admin page (password-protected) that edits the JSON
- A build script reads the JSON and injects values into HTML templates
- More custom work but total control

Option C — Headless CMS (Contentful, Sanity, etc.):

- Overkill for a 6-page static site
- Adds external dependency and potential cost
- Not recommended unless the site grows significantly

Recommendation: Option A (Decap CMS) provides the best balance of capability and simplicity. However, it requires first introducing a templating layer, which is a prerequisite investment.

---

4. AI Chatbot

Current state: `chatbot.js` (321 lines) implements a `PrecisionChatbot` class with:

- Claude API integration via proxy URL
- Fallback keyword-matching for common questions
- Message history tracking
- Typing indicator animation
- Suggestion chips for guided conversation

What needs to happen:

1. **Persona definition** — Write a system prompt that establishes the chatbot's identity, tone, knowledge boundaries, and escalation behavior (e.g., "I can help with general questions, but for specific pricing, please call us at (801) 985-3000").
2. **Knowledge grounding** — Feed the system prompt with accurate company data: services, service area, process, typical project timelines, and FAQ answers.
3. **Backend proxy hardening** — Ensure the proxy handles errors gracefully, has reasonable rate limiting, and logs conversations for quality review.
4. **Fallback behavior** — When the API is unavailable, the keyword fallback should cover the top 10-15 most common

What to avoid: Do not build a quoting engine, a scheduling system, or a document retrieval system into the chatbot. Keep scope to informational Q&A and lead capture.

---

5. Company Voice / Persona

Current state: No documented brand voice. Website copy is professional but not governed by a style guide.

Deliverable: A 1-2 page brand voice document covering:

- **Tone attributes** (e.g., professional, direct, knowledgeable, approachable)
- **Audience definition** (general contractors, builders, architects, homeowners)
- **Vocabulary preferences** (industry terms to use, jargon to avoid)
- **Example phrasings** for common scenarios (describing services, responding to inquiries, social media posts)
- **Chatbot system prompt** derived from the voice document

This is a documentation task, not a development task. It requires 1-2 hours of stakeholder conversation and 2-3 hours of writing.

---

6. Social Media Support

Current state: Facebook and Instagram accounts exist and are linked from the website. No active posting strategy is in place.

Recommended approach:

- Create a monthly content calendar with 3-4 posts per week
- Post categories: completed projects (with photos), industry tips/education, team/culture, seasonal content
- Use the brand voice document for consistent tone
- Tools: Buffer or Later for scheduling (both have free tiers), Canva for simple graphics
- Track engagement monthly and adjust

Content sources that already exist on the site:

- Truss type SVG diagrams (educational content)
- Service descriptions (can be adapted to post format)
- Company history/timeline (anniversary or milestone posts)
- Resources page content (tips and guides)

---

7. SEO Technical Setup

Current state: Meta descriptions and Open Graph tags are present. No structured data, no canonical tags, stale sitemap.

Implementation plan:

TASK	SPECIFICS
JSON-LD: LocalBusiness	Business name, address, phone, hours, service area, geo coordinates
JSON-LD: Service (x3)	One schema per service — roof trusses, floor trusses, custom structural
JSON-LD: FAQPage	Common questions from resources page / chatbot fallback data
Canonical tags	`<link rel="canonical">` on all 6 pages pointing to their canonical URLs
Sitemap update	Update all `lastmod` dates; add `changefreq` and `priority` values
Google Search Console	Verify domain ownership, submit sitemap, monitor indexing
Google Business Profile	Verify/claim listing, ensure NAP consistency with website
Meta optimization	Review and refine meta titles and descriptions for target keywords

Target keywords to research: truss manufacturer Utah, roof trusses Idaho, structural steel Intermountain West, custom trusses [state], floor trusses [city].

---

8. Email-to-Staging Update Workflow

Current state: Not implemented. All site updates require direct code editing.

This is the most complex feature requested. The workflow would be:

```
Client sends email to updates@precisionstructures.net
        |
        v
Email service receives and forwards to Azure Function
        |
        v
AI parses email: identifies what content to change and where
        |
        v
Function generates a proposed edit (HTML diff or content update)
        |
        v
Commits proposed change to a feature branch on GitHub
        |
        v
Developer receives notification, reviews change on staging
        |
        v
Developer approves → merge to develop → visible on staging
        |
        v
Client confirms on staging → merge to master → production
```

Risks:

- AI parsing of free-text emails is inherently unreliable. Ambiguous requests will produce wrong edits.
- Requires a robust review/approval step to prevent bad changes from reaching staging.
- Email threading and attachments add complexity.

Recommendation: Build this only after the client update portal (Feature #3) is in place. The portal provides a structured input mechanism that is far more reliable than email parsing. If email-based updates are still desired, the portal can serve as the "commit target" and the email parser becomes a front-end to the portal rather than a direct code editor.

---

9. Security + Testing

Current state: Security headers are partially configured. No automated testing exists.

Security tasks:

TASK	DETAILS
Content Security Policy	Define allowed script sources, style sources, image sources, font sources, connect sources (for chatbot API). Test thoroughly — CSP can break functionality if too restrictive.
Form CAPTCHA	Add hCaptcha or reCAPTCHA v3 to the contact form. Requires the form backend (Feature #11) to validate the CAPTCHA token server-side.
Input validation	Server-side validation of all form fields in the Azure Function (Feature #11).

Testing tasks:

TASK	DETAILS
Accessibility audit	Run axe-core or WAVE against all 6 pages. Fix critical and serious issues. Target WCAG 2.1 AA compliance.
Link checking	Add a broken-link checker to the GitHub Actions CI/CD pipeline. Fail the build on broken internal links.
HTML validation	Add W3C HTML validator check to CI/CD.
Visual regression	Optional — capture baseline screenshots, compare on each push. Tools: Percy, BackstopJS, or Playwright screenshots with image diff.

---

10. Analytics / Reporting

Current state: No analytics of any kind. Zero visibility into site performance.

Implementation:

1. Create a Google Analytics 4 property
2. Add the GA4 tracking snippet to all 6 pages (in the `<head>` section)
3. Configure the following events:
   - `form_submit` — contact form submission
   - `chatbot_open` — chatbot interaction started
   - `phone_click` — click on phone number link
   - `email_click` — click on email link
   - `gallery_view` — gallery lightbox opened
   - `calculator_use` — estimate calculator interaction

4. Set up a basic Looker Studio dashboard (or use GA4's built-in reports) showing:

- Sessions by source/medium
- Top pages
- Conversion events
- Device breakdown
- Geographic distribution

This should be the first feature implemented after the contact form fix. Every day without analytics is a day without data.

---

11. Contact Form Backend (CRITICAL)

Current state: The contact form in `contact.html` accepts user input across 7 fields (name, email, phone, role, project type, details, file upload) and a submit button. On submission, JavaScript displays a "success" message. **No data is transmitted.** The form is purely cosmetic.

This is the highest-priority fix on this project. A visitor who fills out the form believes they have successfully contacted the company. They will wait for a response that will never come. This damages trust and loses revenue.

Recommended implementation:

```
User submits form on contact.html
        |
        v
JavaScript validates fields client-side (already partially exists)
        |
        v
CAPTCHA token generated (hCaptcha or reCAPTCHA v3)
        |
        v
POST request to Azure Function endpoint (/api/contact)
        |
        v
Azure Function:
  1. Validates CAPTCHA token server-side
  2. Validates and sanitizes all input fields
  3. Handles file upload (store in Azure Blob Storage if present)
  4. Sends notification email to bids@precisionstructures.net via SendGrid
  5. Sends confirmation email to the submitter
  6. Returns success/failure response
        |
        v
JavaScript displays appropriate success or error message
```

Technology choices:

- **Azure Functions (Node.js):** Already in the Azure ecosystem, free tier covers low volume
- **SendGrid:** Free tier allows 100 emails/day, more than sufficient
- **Azure Blob Storage:** For file upload storage (optional — could also attach to email)
- **hCaptcha:** Privacy-friendly CAPTCHA, free tier available

Estimated cost: $0/month at current traffic levels (all services have free tiers).

Implementation Priority Matrix

Organized by recommended execution order:

PHASE	FEATURES	TIMELINE	OUTCOME
Phase 1: Critical Fixes	#11 Contact Form Backend, #10 Analytics, #2 Final Changes (copyright, sitemap)	Week 1	Site is functional and measurable
Phase 2: Visibility	#7 SEO Technical Setup, #5 Company Voice	Week 2	Site is discoverable and has a defined brand identity
Phase 3: Differentiation	#4 AI Chatbot Refinement, #1 Page Review PDFs	Week 3	Site stands out; review workflow established
Phase 4: Growth	#6 Social Media Support, #9 Security + Testing	Week 4	Ongoing presence; quality and security hardened
Phase 5: Scale	#3 Client Update Portal, #8 Email-to-Staging Workflow	Weeks 5-8	Non-technical content management; streamlined updates

Cost Summary (Estimated)

ITEM	ONE-TIME	MONTHLY RECURRING
Azure Functions (free tier)	$0	$0
SendGrid (free tier, 100 emails/day)	$0	$0
Google Analytics 4	$0	$0
Google Search Console	$0	$0
hCaptcha (free tier)	$0	$0
Buffer/Later social scheduling (free tier)	$0	$0
Decap CMS (open source)	$0	$0
Playwright (open source)	$0	$0
Development labor	**Scoped per phase**	—
Total infrastructure cost	**$0**	**$0**

All recommended tools and services operate within free tiers at the expected traffic volume for a regional construction services company. Infrastructure cost becomes non-zero only if traffic exceeds free-tier limits (unlikely in the near term) or if a paid CMS/scheduling tool is selected.

This document should be reviewed alongside the Precision Site Audit. Together, they provide a complete picture of where the site is today and what it takes to get it where it needs to be.